

程序设计90

张宏斌 副教授

复习

- 复合语句
- 条件表达式
- 分支结构
 - 单分支
 - 双分支
 - 多分支
 - 嵌套的分支结构
 - 三元运算符
- 循环结构
 - while循环
 - for循环

复习

- 循环语句中的continue、break、else子语句
- 三种和循环相关的内置函数：
 - zip()函数
 - map()函数
 - enumerate()函数

第九章 异常处理

- 1 熟悉异常的概念
- 2 掌握异常处理语句结构
- 3 了解 raise 语句的用法
- 4 掌握 assert 语句的用法
- 5 灵活运用 try 语句解决实际异常问题

异常概述

- 当程序发生**错误**时，Python 中的**异常处理**会自动被**触发**。这样可以跳到异常处理器中，对异常进行相应的处理
- 如果程序忽略发生的错误，Python 将启动默认的异常处理：**停止程序并打印错误信息**
- 通过 **try 语句**可以捕获并处理发生的异常情况，这样程序在执行完 **try 语句**后将继续执行后面的代码，**而不是停止程序**



try/except/else语句

- 程序在运行过程中，可能遇到不可控制的错误：

```
>>> a = 128
```

```
>>> b = 0
```

```
>>> print(a/b)
```

Traceback (most recent call last):

File "<pyshell#2>", line 1, in <module>

print(a/b)

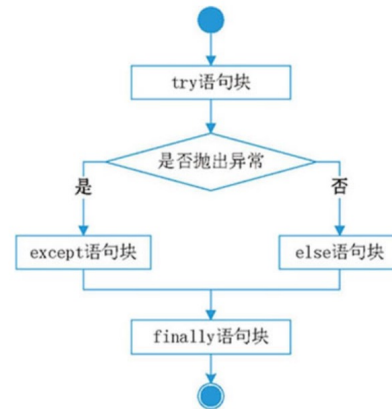
ZeroDivisionError: division by zero



try/except/else语句

- 若要捕获和处理异常，需要将可能发生错误的代码包装在 **try** 语句中，并通过 **except** 子句捕捉和处理异常，其语法格式如下所示：

```
try:
    statements1
except name1:
    statements2
except (name2, name3):
    statements3
except name4 as err:
    statements4
except:
    statements5
else:
    statements6
```



```
>>> a = 128
>>> b = 0
>>> try:
...     c = a/b
... except ZeroDivisionError:
...     print("除数不能为0! ")
... else:
...     print(f"商为{c}")
...
除数不能为0!
```

try/except/else语句：案例

- 定义一个股票信息构成的列表：股票代码、股票名称、交易价格和交易数量，计算交易额。当发生异常时，返回的交易额为 0

```
>>> stock_list = ['300905', '宝丽迪', 34.95, 216500]
>>> try:
...     tran_amount = stock_list[2] * stock_list[3]
... except IndexError:
...     print("列表中应包含股票代码、名称、价格、交易量！")
...     tran_amount = 0
... except:
...     print("发生了其他错误！")
...     tran_amount = 0
... else:
...     print("一切正常，没有错误！")
...
...
一切正常，没有错误！
>>> print(f"{stock_list[1]}的交易额为{tran_amount}")
宝丽迪的交易额为7566675.0
```

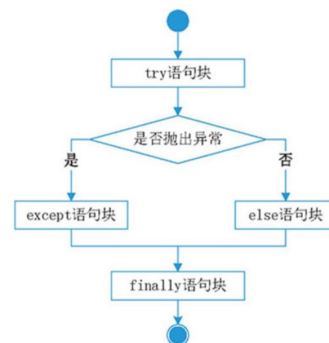
```
>>> stock_list = ['300905', '宝丽迪', 34.95]
>>> try:
...     tran_amount = stock_list[2]*stock_list[3]
... except IndexError:
...     print("列表中应包含股票代码、名称、价格、交易量！")
...     tran_amount = 0
... except:
...     print("发生了其他错误！")
...     tran_amount = 0
... else:
...     print("一切正常，没有错误！")
...
...
列表应包含股票代码、名称、价格、交易量！
>>> print(f'{stock_list[1]}的交易额为{tran_amount}')
宝丽迪的交易额为0
```

```
>>> stock_list = ['300905', '宝丽迪', 34.95, '216500']
>>> try:
...     tran_amount = stock_list[2]*stock_list[3]
... except IndexError:
...     print("列表中应包含股票代码、名称、价格、交易量！")
...     tran_amount = 0
... except:
...     print("发生了其他错误！")
...     tran_amount = 0
... else:
...     print("一切正常，没有错误！")
...
...
发生了其他错误！
>>> print(f'{stock_list[1]}的交易额为{tran_amount}')
宝丽迪的交易额为0
```


try/finally语句

- 如果希望能保证无论程序是否发生异常，都可以执行一些代码。此时可以利用 **try/finally** 的形式来实现：

```
try:  
    statements1  
finally:  
    statements2
```



- finally** 子句是 **try** 语句结束前执行的最后一项任务。不论 **try** 语句是否触发异常，都会执行 **finally** 子句。如果执行 **try** 子句期间触发了某个异常，则某个 **except** 子句应处理该异常。如果该异常没有 **except** 子句处理，在 **finally** 子句执行后会被重新触发

```
>>> x = 12  
>>> y = 3  
>>> try:  
...     result = x / y  
... except ZeroDivisionError:  
...     print("除数不能为0!")  
... else:  
...     print("商为：", result)  
... finally:  
...     print("运行最后一句！")  
...  
...  
商为： 4.0  
运行最后一句！
```

```
x = 12
y = 0
try:
    result = x / y
except ZeroDivisionError:
    print("除数不能为0!")
else:
    print("商为: ", result)
finally:
    print("运行最后一句! ")
```

除数不能为0!
运行最后一句!

```
>>> x = '12'
>>> y = 3
>>> try:
...     result = x / y
... except ZeroDivisionError:
...     print("除数不能为0!")
... else:
...     print("商为: ", result)
... finally:
...     print("运行最后一句! ")
...
...
```

运行最后一句!

Traceback (most recent call last):

File "<pyshell#52>", line 2, in <module>

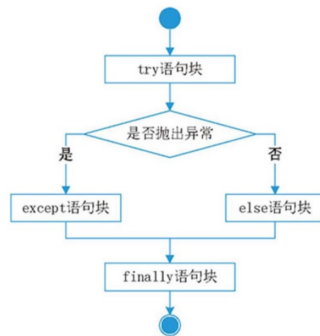
result = x / y

TypeError: unsupported operand type(s) for /: 'str' and 'int'

完整try语句

- 在 **try** 语句中可以混合使用 **except**、**else** 和 **finally** 语句：

- 在try 语句中必须至少有一个 **except** 或一个 **finally**
- 如果有的话，必须按照
try → except → else → finally 的顺序出现



```
try:
    statements1
except name1:
    statements2
except (name2, name3):
    statements3
except name4 as err:
    statements4
except:
    statements5
else:
    statements6
finally:
    statements2
```



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



AACSB
ACCREDITED



ASSOCIATION
AMBA
ACCREDITED

raise语句

- 除了由运行期间数据不符合语法规则引起的异常外，也可以通过 **raise** 语句主动触发异常
- 通过 **raise** 语句触发的异常也会被 **try** 语句的 **except** 分句捕获和处理

```
❶ try:
❷     raise IndexError
❸ except IndexError:
❹     print('触发了 IndexError。')
❺ print('程序继续执行。')
```

触发了 IndexError。
程序继续执行。



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



AACSB
ACCREDITED



ASSOCIATION
AMBA
ACCREDITED

assert语句

- 在程序开发期间，有时会使用 **assert** 语句验证程序运行状况。这通常是出于调试程序的目的。
assert 语句可以看作条件式的 **raise** 语句
- **assert** 语句，又称**断言语句**，可以看做是功能缩小版的 if 语句，它用于判断某个表达式的值，如果值为真，则程序可以继续往下执行；反之，Python 解释器会报 **AssertionError** 错误

```
mathmark = int(input())  
# 断言数学考试分数是否位于正常范围内  
assert 0 <= mathmark <= 100  
# 只有当 mathmark 位于 [0,100]范围内，程序才会继续执行  
print("数学考试分数为：", mathmark)
```

assert语句

- **assert**语句的格式为：
assert <表达式>, <错误提示语>

```
>>> x = 12  
>>> y = 0  
>>> assert y != 0, "除数不能为0! "  
Traceback (most recent call last):  
  File "<pyshell#55>", line 1, in <module>  
    assert y != 0, "除数不能为0! "  
AssertionError: 除数不能为0!
```

assert语句

```
x = 12
```

```
y = 0
```

```
try:
```

```
    assert y != 0, "除数不能为0! "
```

```
    print(x/y)
```

```
except AssertionError as error:
```

```
    print(error)
```

总结

- 异常处理的必要性
- try/except/else/finally语句
- raise语句
- assert语句

期中测试

- 客观题为闭卷考试，请大家不要带任何电子产品（包括电子表）、书籍、资料等
- 根据学校制度，考试作弊会被开除！请大家遵守考试纪律
- 理论题答题时间30分钟